

Дисциплина: «Введение в глубокое обучение»

Общая информация

Длительность	Три недели
Семестр	2-й семестр
Аттестация	Экзамен

Авторы и преподаватели

Дмитрий Коробченко

- NVIDIA, Deep Learning R&D-инженер.
- Исследователь и разработчик в областях машинного обучения, нейронных сетей, компьютерного зрения и обработки сигналов.
- Занимался задачами, связанными с медицинскими изображениями, распознаванием, обработкой изображений, 3D-анимацией.
- Время от времени читает вводные научно-популярные лекции по Deep Learning.
- Преподаватель Skillfactory по нейронным сетям, AI-разработке, ML и DL.

Константин Некрасов

- АО «Газпромбанк», директор по разработке ML-моделей.
- Ведёт проекты в области машинного обучения для решения прикладных задач финтеха.
- Эксперт в глубоком обучении, в частности в направлениях обработки естественного языка (NLP).
- Разрабатывает и внедряет ML-модели в продуктовую среду.
- Преподаёт дисциплины по Data Science, Data Engineering, Machine Learning, DevOps и Web Development в ведущих вузах страны: МИФИ, МФТИ, ВШЭ.

О дисциплине

Дисциплина посвящена практическому освоению методов глубокого обучения. Вы реализуете полный цикл разработки модели машинного обучения — от загрузки и анализа данных до построения, обучения и применения модели на пользовательских данных. Особое внимание уделяется сравнению подходов и инструментов, используемых в современных фреймворках глубокого обучения, таких как PyTorch и TensorFlow.

К 2026 году глубокое обучение стало ключевой технологией, лежащей в основе большинства интеллектуальных систем — от генеративных моделей и цифровых ассистентов до систем компьютерного зрения и анализа данных в промышленности. Компании ожидают от специалистов не поверхностного знакомства с инструментами, а глубокого понимания принципов работы моделей, умения анализировать их поведение и адаптировать под реальные задачи.

Вы научитесь проводить исследовательский анализ данных, разрабатывать и обучать нейронные сети, оценивать качество моделей и анализировать ошибки. Важная часть дисциплины — интерпретация результатов, выявление ограничений моделей и применение методов улучшения качества, включая регуляризацию. Также рассматриваются вопросы воспроизводимости экспериментов и корректного оформления результатов работы.

Цель дисциплины

Сформировать практические навыки разработки и анализа моделей глубокого обучения, а также умение реализовывать полный цикл решения задачи классификации — от подготовки данных до инференса и интерпретации результатов.

В рамках дисциплины вы:

- научитесь загружать и подготавливать данные для задач компьютерного зрения;
- освоите построение и обучение нейронных сетей в PyTorch и TensorFlow;
- научитесь проводить исследовательский анализ данных (EDA) и визуализацию;
- освоите методы оценки качества моделей и анализа ошибок (включая confusion matrix);
- изучите подходы к сравнению моделей и фреймворков;
- получите опыт улучшения моделей и борьбы с переобучением;
- реализуете инференс модели на пользовательских данных.

Пререквизиты и постреквизиты

Что нужно знать для прохождения дисциплины:

- Основы программирования на Python.
- Базовые знания линейной алгебры и основ машинного обучения.

Что можно будет изучать после прохождения дисциплины:

- Продвинутые методы глубокого обучения.
- Компьютерное зрение.
- Обработка естественного языка.
- Исследовательские направления в машинном обучении.

Содержание дисциплины

№	Раздел	Образовательные результаты модуля
1	Модуль 1. Введение в нейронные сети	► Машинное обучение и типы данных. ► Нейронные сети. ► Как обучить нейронную сеть. ► Задача компьютерного зрения. ► Где взять ядро свёртки. ► Что такое Deep Learning. Типы слоёв. ► Обработка последовательностей. ► От распознавания к синтезу. ► Составлятельные сети
2	Модуль 2. Фреймворки для глубокого обучения	► Граф вычислений. ► MLP. ► TensorFlow. ► Пример обучения. ► Детали градиентного спуска: цепное правило. ► Граф вычисления производных. ► Пример одного нейрона. ► Цепное правило и граф производных
3	Модуль 3. Основы работы с PyTorch	► Введение в PyTorch и тензоры. ► Работа с устройствами (CPU/GPU). ► Autograd: автоматическое дифференцирование. ► Построение моделей: torch.nn.Module. ► Функции активации, функции потерь и регуляризация. ► Загрузка данных: Dataset и DataLoader. ► Оптимизаторы и сохранение моделей. ► Валидация и режимы обучения

Практика

Практическая подготовка по дисциплине происходит в формате синхронных онлайн-занятий и асинхронной самостоятельной работы на образовательной платформе.

Синхронные занятия проводятся в формате практико-ориентированных онлайн-семинаров с разбором прикладных кейсов и архитектурных решений.

Асинхронная практика включает интерактивные задания: тесты (выбор одного или нескольких ответов), задания с вводом ответа и написание кода с автоматической проверкой. Упражнения направлены на освоение инструментов и методов глубокого обучения.

- По завершении дисциплины выполняется семестровое задание — **итоговое тестирование**. Срок выполнения — до трёх недель после окончания курса. Во время сессии задание сдать уже нельзя.
- Итогом дисциплины является **проект (сессионное задание)** — кейс «Классификация рукописных символов с использованием глубокого обучения». Проект сдаётся в течение двух недель после завершения дисциплины либо в период сессии.

Оценивание

Оценка за дисциплину суммируется из оценок за выполнение итогового тестирования (максимальное количество баллов — 50) и сессионного задания, за которое вы также сможете получить максимально 50 баллов.

Шкала перевода баллов в оценку

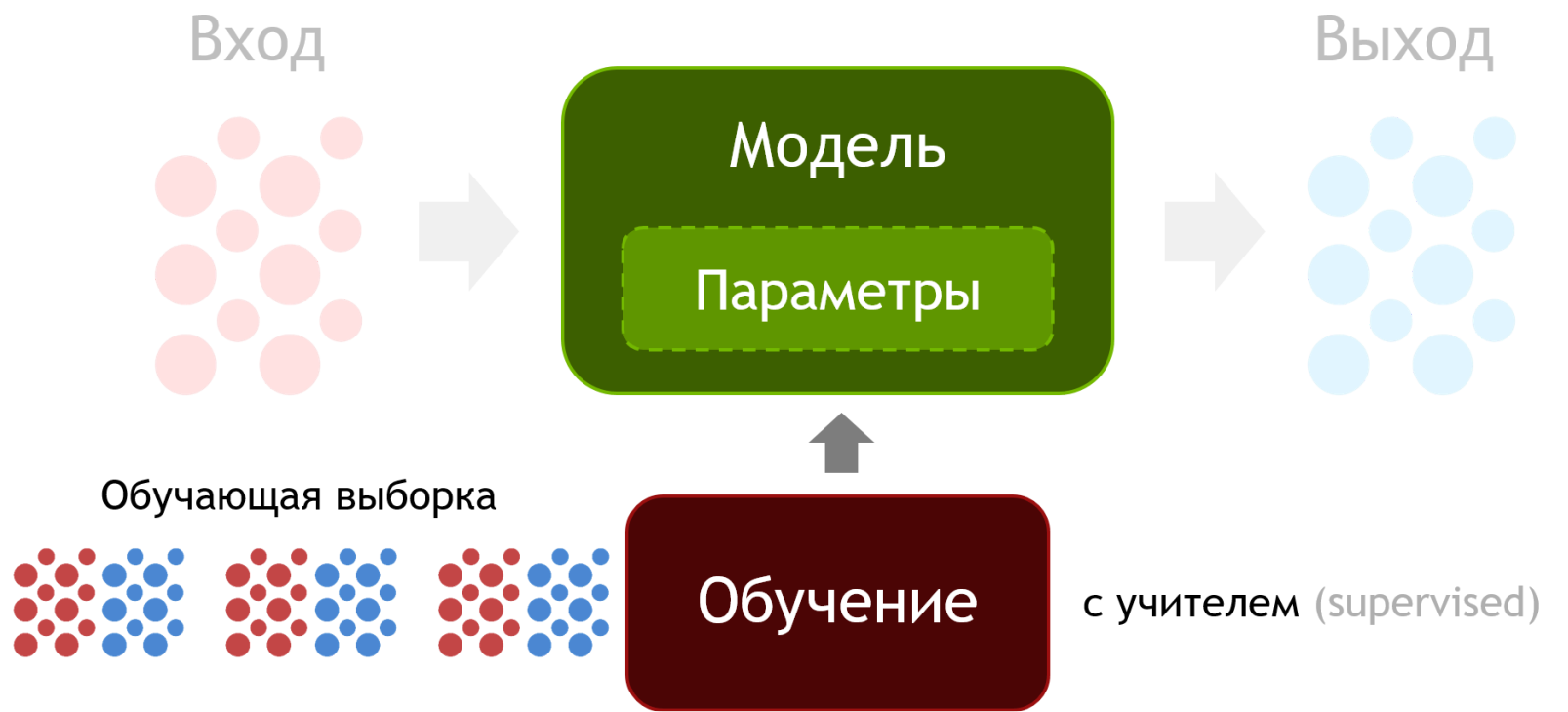
Оценка за дисциплину выставляется по шкале МИФИ

Баллы	Буквенная оценка	Оценка
90–100	A	Отлично
85–89	B	Хорошо
75–84	C	
70–74	D	
65–69	E	Удовлетворительно
60–64		
59 и ниже	F	Неудовлетворительно



Прежде чем переходить к Deep Learning (глубокому обучению), актуализируем информацию по машинному обучению.

Машинное обучение с высоты птичьего полёта можно рассмотреть как некоторое отображение входных данных в выходные данные. Как правило, этим занимается некоторая модель, которая зависит от заданных параметров. Эти параметры характеризуют то, как модель себя ведёт, как она предсказывает выходные значения по входным значениям. Подстройка этих параметров и есть машинное обучение.



Классическое обучение с учителем

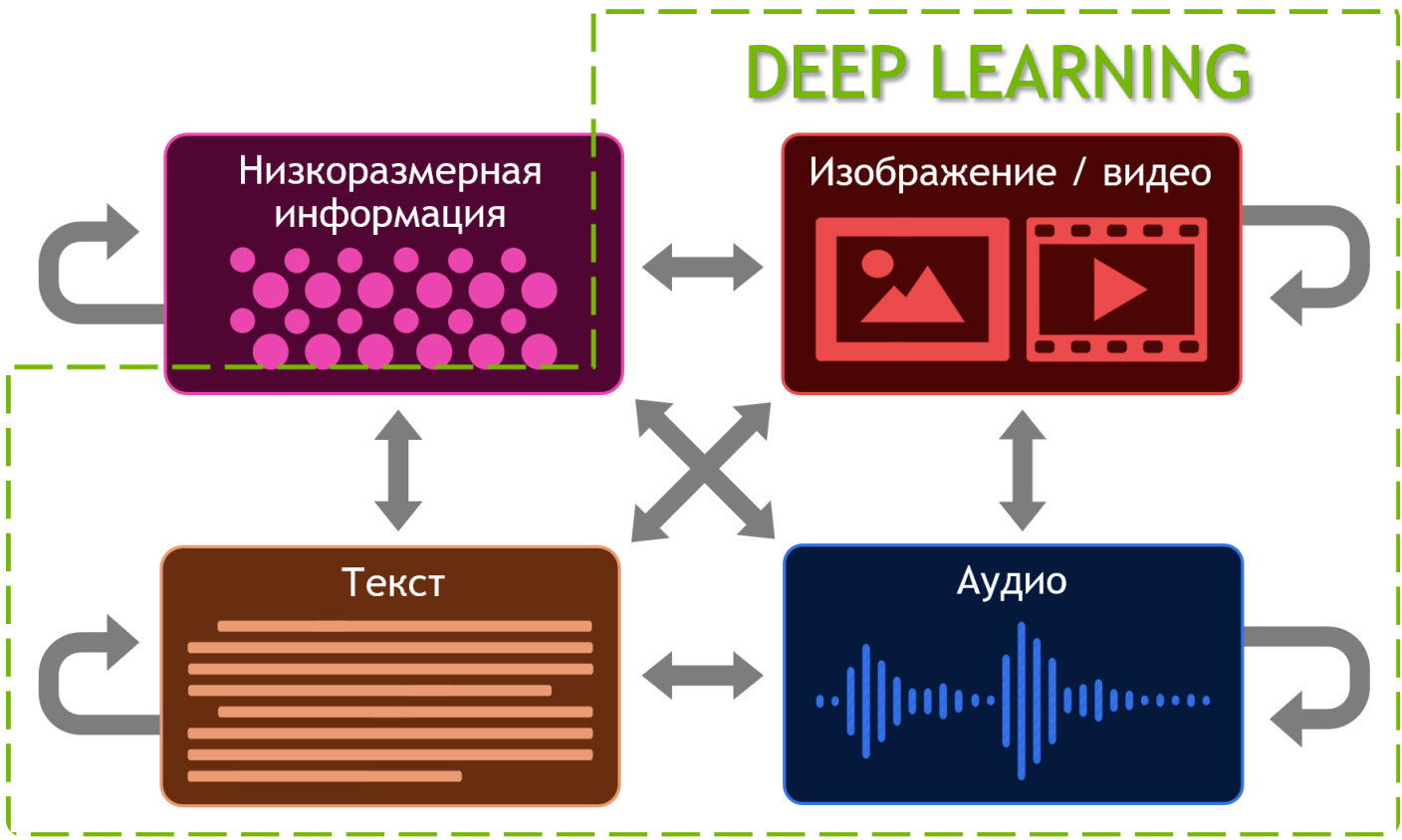
Как правило, в машинном обучении используется обучение с учителем (supervised learning), в котором у нас есть база данных, состоящая из пар объект – ответ. Мы показываем нашей модели эти пары одну за одной и корректируем параметры таким образом, чтобы на новых примерах наши модели предсказали хорошие правильные ответы.

Какие типы данных могут использоваться в таких задачах?

1. Низкоразмерная информация.
Например, это могут быть векторы, отвечающие за характеристики пользователей (рост, вес, возраст и т.д.), и мы хотим сделать их классификацию: то есть отображать низкоразмерный вектор в другой низкоразмерный вектор или скаляр.
2. Изображение или видео.
Это более сложный тип данных. В качестве примера здесь можно рассмотреть задачу компьютерного зрения: отображение визуальной информации в низкоразмерную высокоуровневую информацию.
3. Текст.
Пример: обработка текста и его отображение в качестве низкоразмерной высокоуровневой информации.
4. Аудио и звуковые сигналы.
Всем знакомый пример: распознавание речи и перевод аудио в текст.

Мы можем делать практически любое отображение из любого типа данных в любой тип данных. При этом на каждое такое отображение есть соответствующая задача и методы её решения. Большинство этих задач наилучшим образом решаются с помощью нейронных сетей или технологии Deep Learning.

Важно понимать, что нейронные сети не всегда являются лучшим решением. Например, отображение низкоуровневой информации в саму себя лучше реализовать с помощью других алгоритмов (случайный лес, градиентный бустинг и др.). Но в случае с высокоразмерными сложными данными (изображения, звуки, тексты) очень хорошо работают именно нейронные сети.



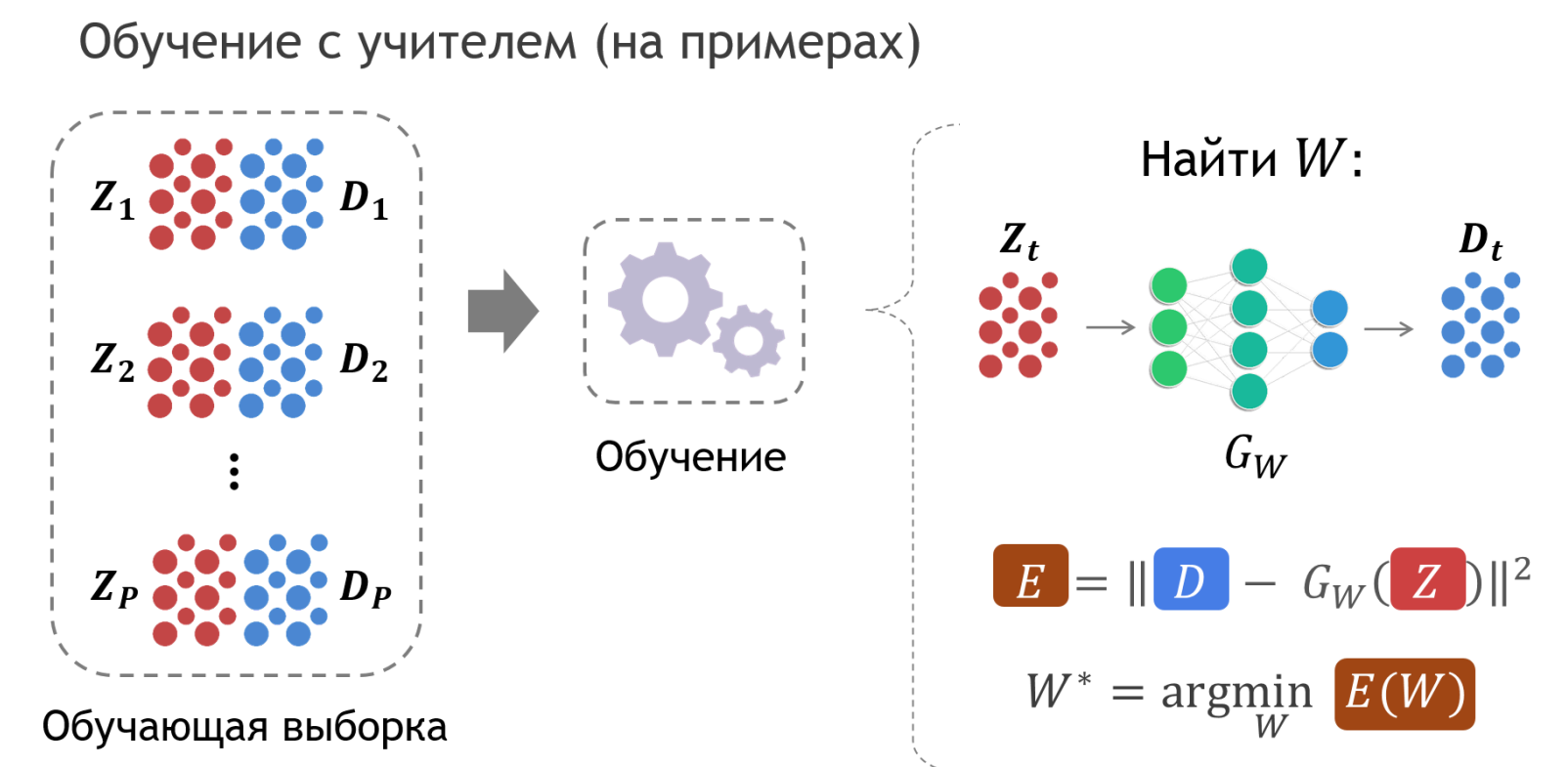


SKILLFACTORY

Чаще всего нейронные сети обучаются с учителем на размеченных данных. Для этого нам нужна обучающая выборка, которая состоит из пар входной объект — выходной объект. Мы подаём эту обучающую выборку в процесс обучения, который состоит в том, чтобы найти такие параметры модели W , чтобы наша нейронная сеть предсказывала правильно те самые ответы, которые мы уже знаем.

Таким образом мы решаем задачу минимизации или задачу оптимизации: мы минимизируем ошибку на тех примерах, которые есть в нашей обучающей выборке.

Ошибку можно записать по-разному:



Как решать задачу оптимизации?

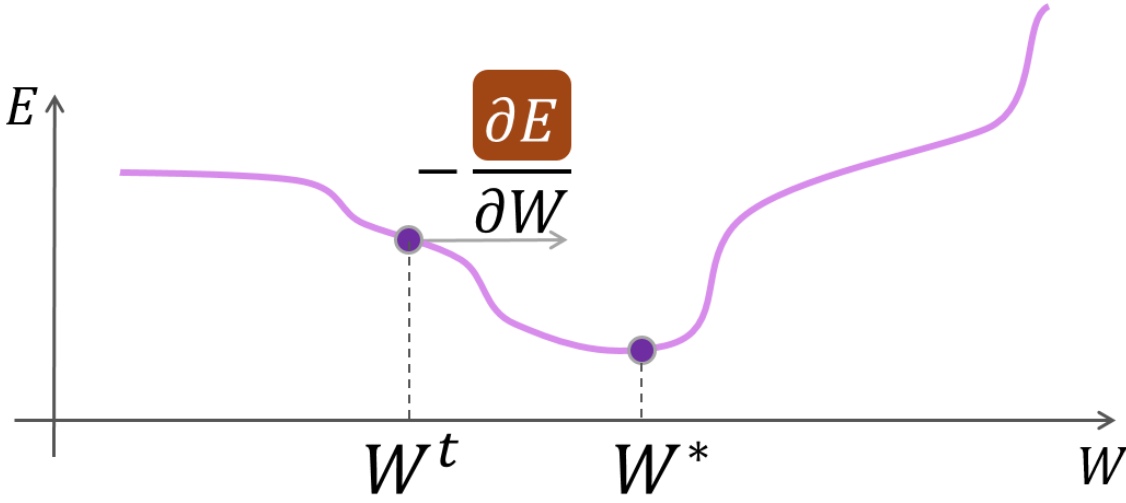
Есть различные способы. В теории оптимизации есть такой известный алгоритм как **градиентный спуск**. Смысл объясним на примере.

Допустим, у вас есть какая-то функция (здесь представлена одномерная функция, но в общем случае она может быть и многомерной), и вы хотите найти её минимум. Вы стоите в некоторой точке, и вам нужно понять, в какую сторону вам с этой точки нужно сдвинуться, чтобы приблизиться к минимуму. Есть вектор, который называется **градиент**. И этот вектор смотрит в сторону возрастания функции. Поэтому градиент со знаком минус смотрит в сторону убывания функции.

Подобный алгоритм предлагает двигаться в сторону антиградиента и таким образом приближаться к локальному минимуму.

Итак, мы вычислили градиент, и с некоторым **параметром α** , который ещё называют **Learning rate** (скорость обучения), мы этот антиградиент прибавляем к нашим текущим весам, и так получается итерационное движение к минимуму.

Градиентный спуск



Итерационный процесс

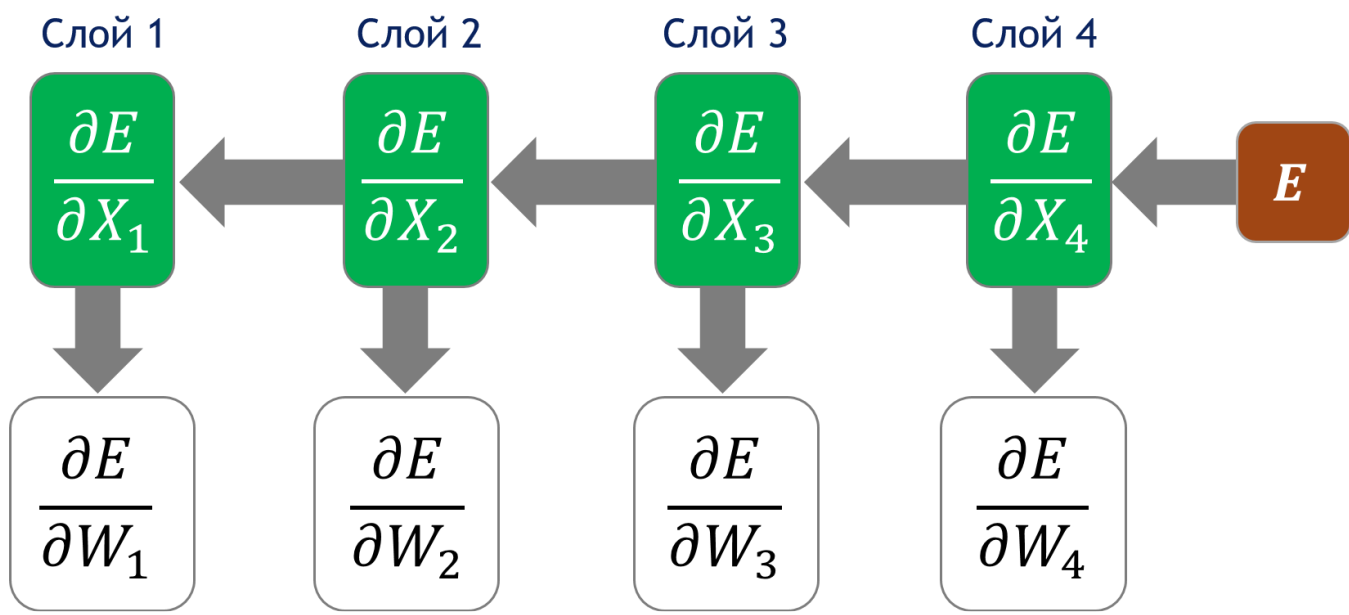
$$W^{t+1} = W^t - \alpha \frac{\partial E}{\partial W}(W^t)$$

В случае нейронных сетей используется небольшая модификация — **стохастический градиентный спуск**. Отличие от предыдущего градиентного спуска в том, что мы вычисляем градиент не сразу на всех образцах нашей выборки, а только на одном образце за одну итерацию или на группе образцов.

Как вычислить градиент?

Это вектор или даже некоторый тензор, размерность которого совпадает с размерностью всех наших параметров обучаемых.

Обратное распространение ошибки



Градиент может особенно зависеть от слоев, которые далеки от той самой ошибки, от которой мы считаем градиент. Например, у нас ошибка зависит от параметров первого слоя очень сложным образом, и, тем не менее, мы всё равно можем вычислить градиент с помощью **алгоритма обратного распространения ошибки**, который заключается в том, что мы используем правила дифференцирования сложной функции.

Нейронная сеть, даже если она представляет собой сложную функцию — на самом деле просто композиция каких-то маленьких простых вещей. Например, умножили на матрицу, прибавили вектор, взяли поэлементную матрицу и так далее. Используя такое дифференцирование сложной функции, можно узнать, как наша ошибка зависела от параметров второго слоя через это цепное правило. Именно таким образом можно вычислить градиент по любому весу и по всем весам. Это обратное распространение ошибки, или ещё его называют **backpropagation**.

Нейронные сети могут применяться для решения различных типов задач:

- классификация;
- регрессия;
- задачи генерации данных.

Они также широко используются в различных областях:

- компьютерное зрение;
- обработка текста;
- обработка речи.

SKILLFACTORY

Задача компьютерного зрения состоит в отображении визуальной информации (изображения или видео) в высокоуровневую, семантическую информацию.

В классификации изображений на входе находится картинка, на выходе — метка класса.

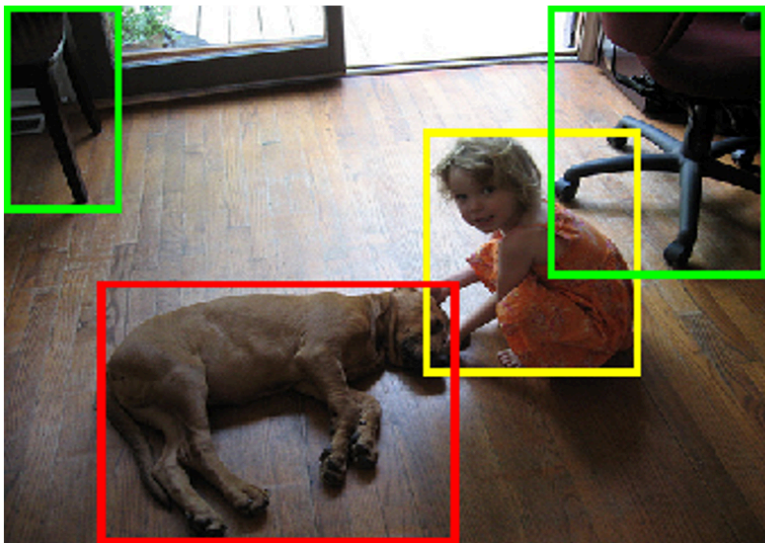
При детектировании и локализации объектов, помимо классификации, даётся ограничивающий прямоугольник, или локализация, где тот или иной объект присутствует на изображении.

Классификация



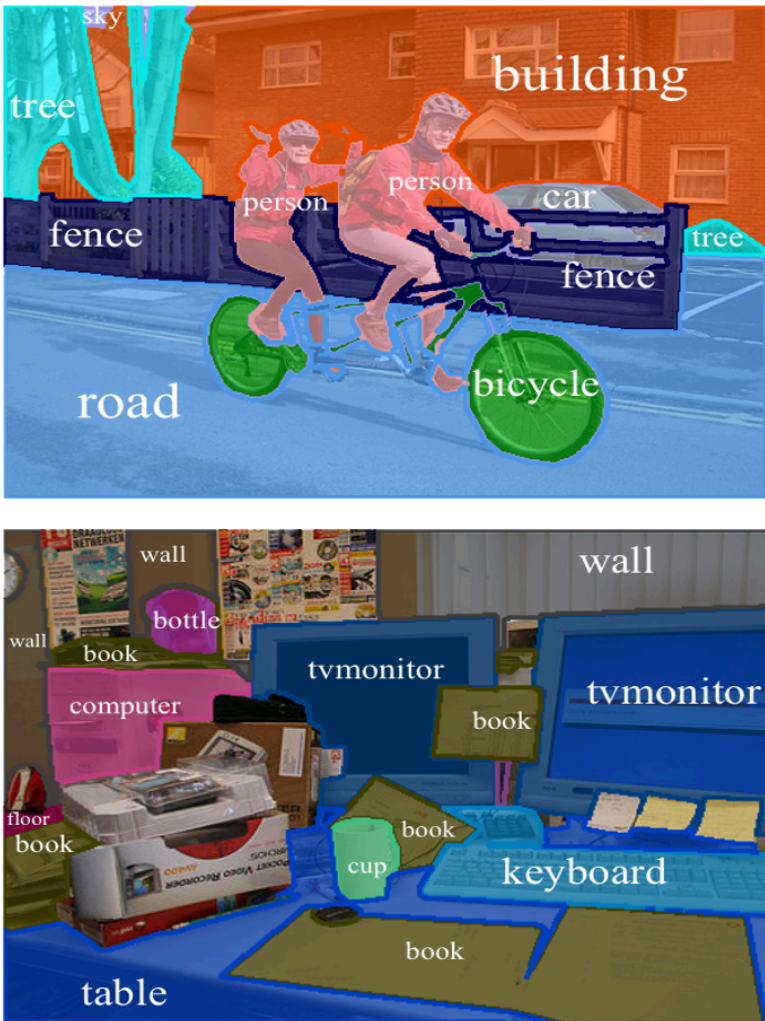
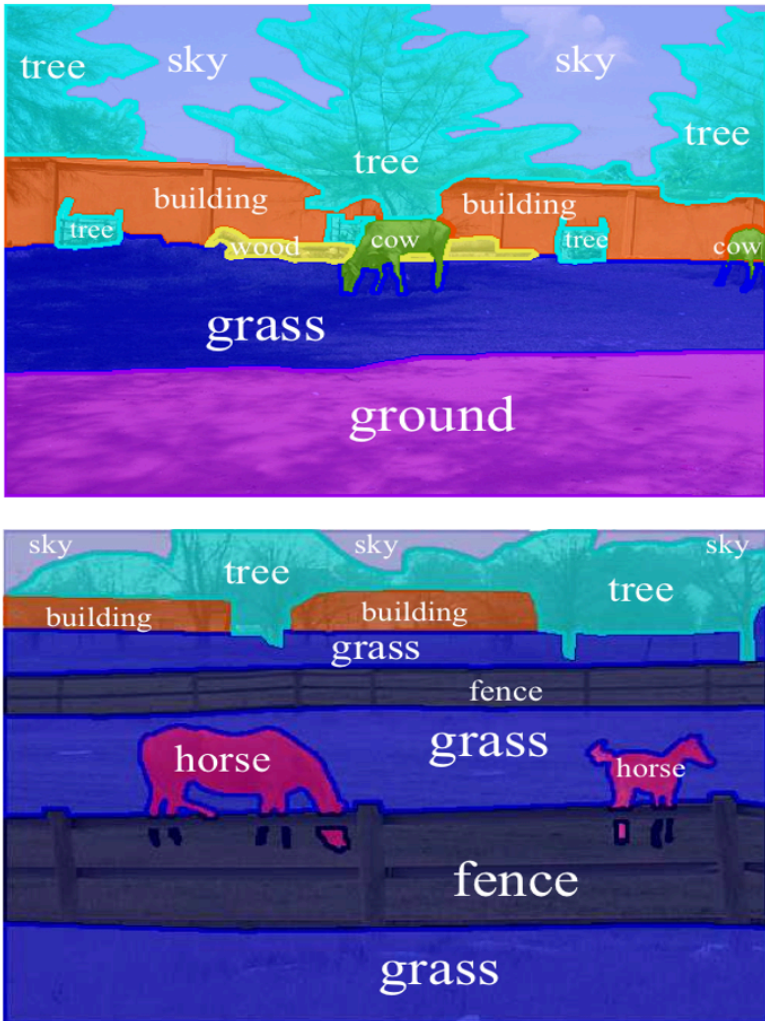
“Котик”

Детектирование объектов



- Человек
- Собака
- Стул

Семантическая сегментация — более сложная задача, в ней классифицируется каждый пиксель изображения, или выделяются некоторые сегменты на изображении, и каждому сегменту присваивается какая-то метрика.

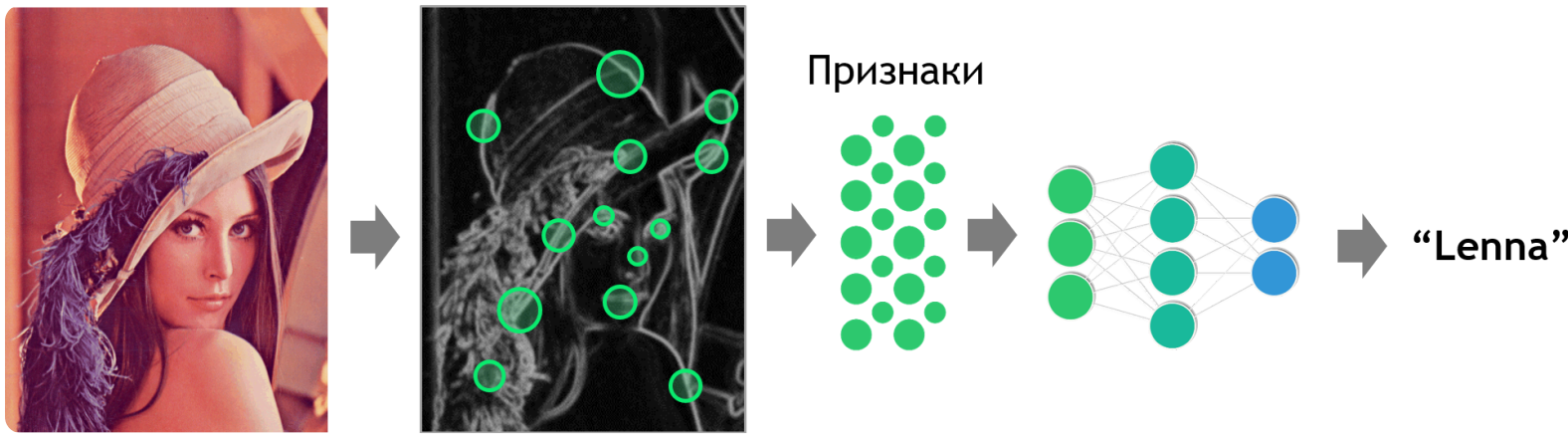


Семантическая сегментация

Основная трудность этих задач в том, что картинка или изображение представляет собой огромное количество неструктурированной информации, к которой непонятно, как подойти: это просто какие-то числа RGB. В машинном зрении ещё до прихода нейронных сетей использовались признаки. Признаки — это то, что описывает изображения. На картинке находятся какие-то особенности, они записываются в виде уже осмысленного высокоуровневого вектора и дальше к этому вектору применяется алгоритм классификации и получается ответ.

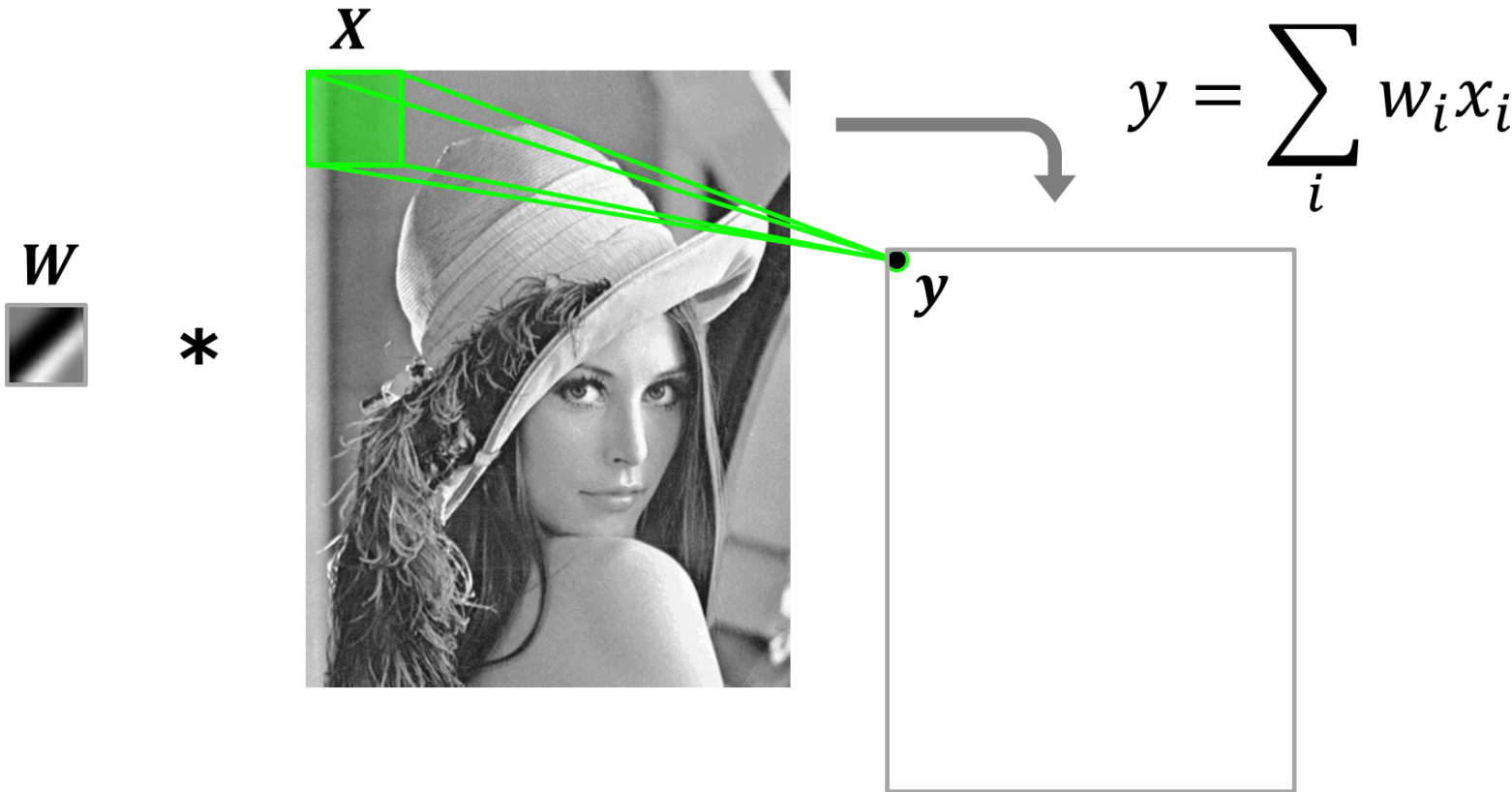
В итоге решение задачи сводится к двум частям:

1. извлечение признаков;
2. подача признаков в алгоритм машинного обучения.



До появления машинного зрения все признаки создавались вручную специальными инженерами, которые анализировали изображение и вручную кодировали то, как признаки должны выглядеть.

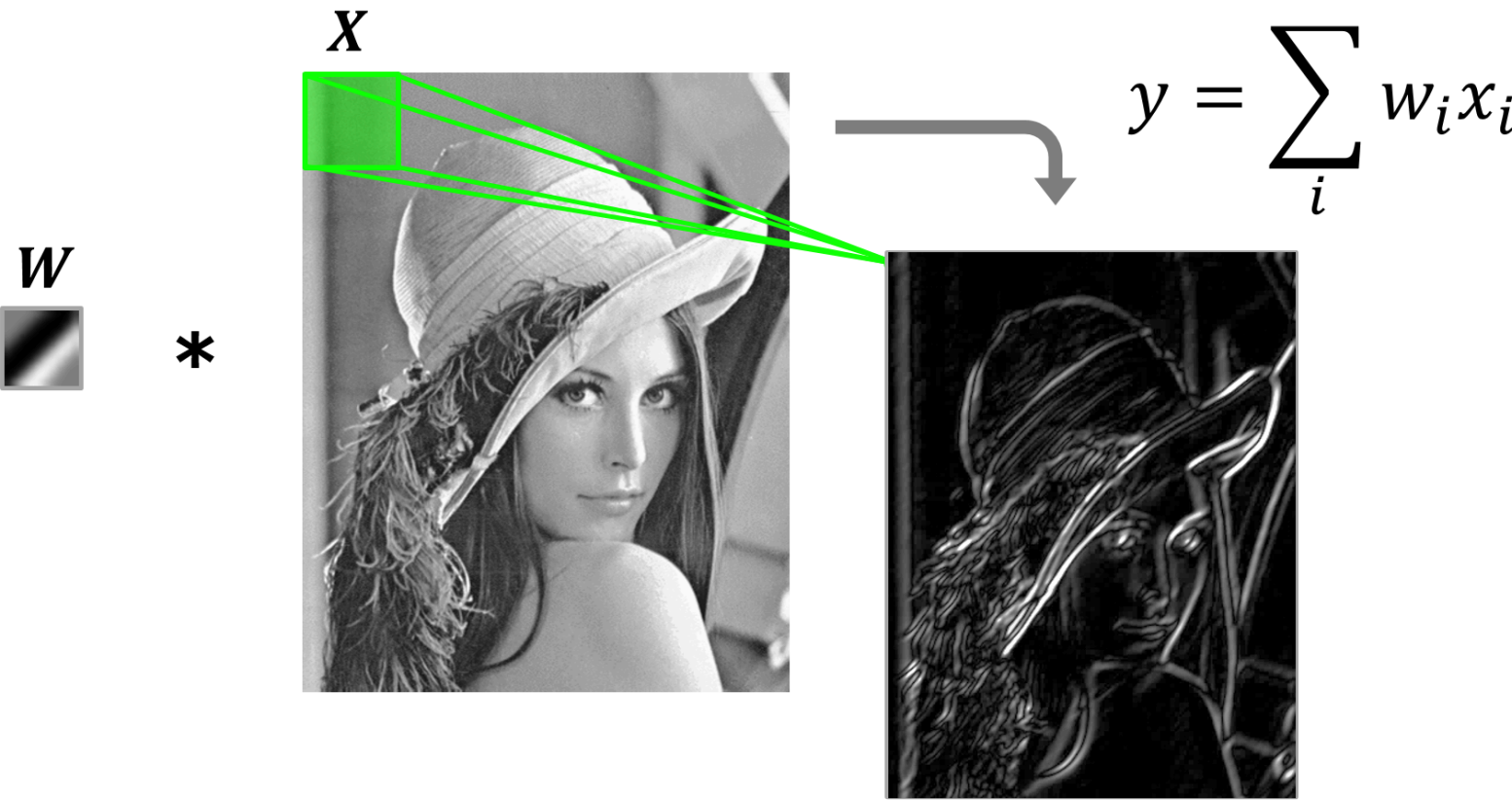
Свёртка — это способ извлечения признаков.





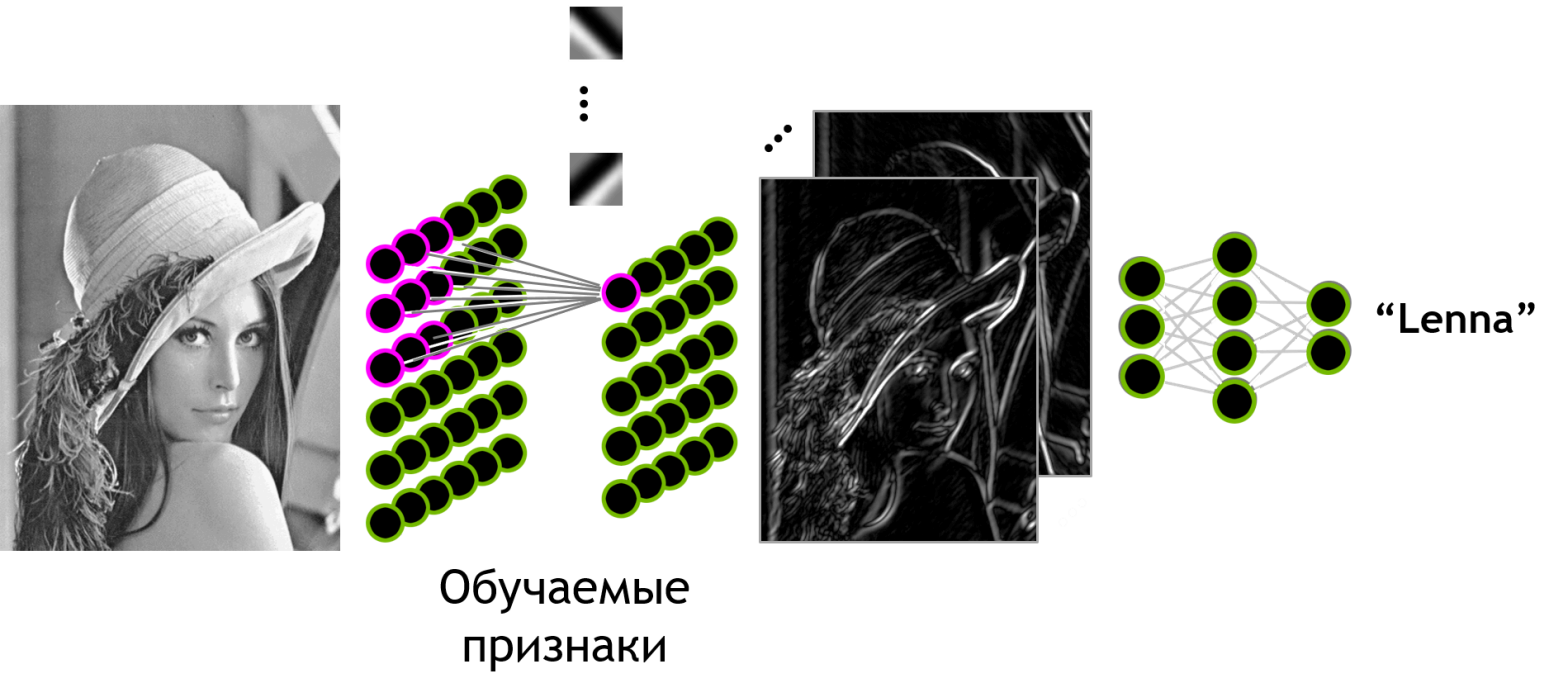
Идея свёрточных нейронных сетей состоит в том, чтобы использовать свёртку как специальный тип слоя в нейронной сети.

Обратимся к предыдущему примеру:



Посмотрим на свёртку: здесь видно, что у нас используется некоторая линейная операция для вычисления значения в выходной матрице. Такая же линейная операция использовалась нами в нейронных сетях.

Вернёмся:



Разница в том, что нейроны уже связаны в соседних тензорах. Не каждый с каждым, а только какая-то группа нейронов во входном слое связана с одним нейроном в выходном слое. Именно так и делается в свёрточных нейронных сетях.

Веса в ядре свёртки получаются с помощью оптимизации. Теперь они соответствуют связям между нейронами. Таким образом, мы получили автоматически обучаемые ядра свёрток или, другими словами, **обучаемые признаки** (в отличие от тех признаков в компьютерном зрении, которые использовались ранее, когда люди задавали их вручную).

Основная идея глубокого обучения — не просто изучение признаков, а извлечение их иерархии: извлечение признаков вышестоящего уровня в пространстве нижестоящего.

7/17

Что такое Deep Learning? Типы слоёв

текущий урок

SKILLFACTORY

Deep Learning — это обучение иерархии признаковых представлений.



Deep Learning (глубокое обучение) — это подход в машинном обучении, основанный на использовании многослойных параметризованных моделей (обычно нейронных сетей), которые автоматически обучают иерархические представления данных путём оптимизации функции потерь с использованием градиентных методов.

Что здесь важно:

- Тут есть признаки: сначала из входных данных извлекаются признаки, и дальше работа идёт уже с ними .
- Тут есть обучаемые признаки: мы делаем так, чтобы нейронная сеть их учила автоматически.
- Тут не просто один слой признаков — здесь иерархия признаков.

Какие типы слоёв бывают в таких сетях?

- свёрточный слой;
- понижение размерности (Pooling):
 - локальное усреднение;
 - локальный максимум;
- полносвязный слой (Fully-connected).

← Предыдущий урок

Следующий урок →

8/17

Обработка последовательностей

текущий урок



Обработка последовательностей в нейронных сетях заключается в том, что модель учитывает порядок элементов и использует информацию о предыдущих шагах при обработке текущего элемента.

В классическом подходе для этого применялись **рекуррентные нейронные сети (RNN)**, в которых используется скрытое состояние (hidden state), позволяющее сохранять информацию о предыдущих элементах последовательности.

Более продвинутые варианты рекуррентных сетей:

- **LSTM (Long Short-Term Memory)**
- **GRU (Gated Recurrent Unit)**

Эти архитектуры были разработаны для того, чтобы лучше сохранять долгосрочные зависимости в данных и частично решать проблему затухания градиента.

Однако в современных задачах обработки последовательностей чаще используют **архитектуры на основе трансформеров (Transformers)**, которые позволяют эффективно учитывать зависимости между элементами последовательности без рекуррентных вычислений.

Тем не менее рекуррентные сети остаются важным базовым подходом и используются в ряде задач и образовательных целей.

Где применяются модели для обработки последовательностей:

- Предиктивный ввод текста (умные клавиатуры).
- Анализ текста и комментариев.
- Машинный перевод.
- Чат-боты и диалоговые системы.
- Распознавание речи.
- Генерация описаний изображений и мультимодальные задачи.

<

Предыдущий урок

Следующий урок

>

SKILLFACTORY

Синтез — это обратная задача, то есть перевод информации высокого уровня в информацию низкого уровня.

На примере человека — восприятие и творчество:



На примере работы техники— распознавание и синтез:





Состязательная сеть (англ. Generative Adversarial Networks, GAN).

Идея состоит в том, чтобы построить вторую сеть, которая называется дискриминатор. Это просто бинарный классификатор: на входе — картинка, а на выходе — ответ с предположением, реальная картинка или синтезированная.



Важный момент: генератор должен учиться обманывать дискриминатор. Они учатся параллельно, играя в антагонистическую игру, и в итоге мы получаем идеальный генератор и идеальный дискриминатор.

Примеры применения:

- ▶ синтез изображений;
- ▶ преобразование текста в изображение;
- ▶ отображение изображения в изображение;
- ▶ отображение видео в видео;
- ▶ синтез речи (Wavenet);
- ▶ аудио в видео.

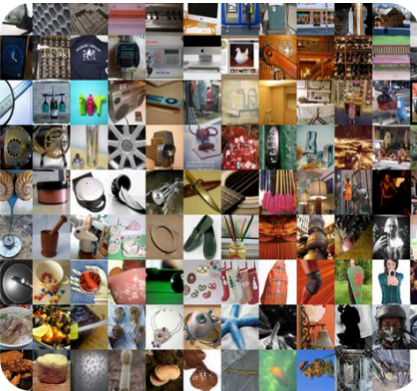
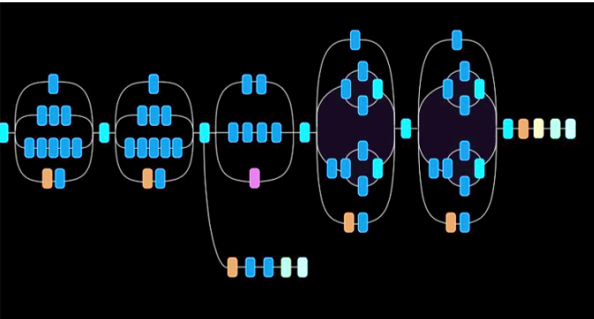
← Предыдущий урок

Следующий урок >



Причины успеха глубокого обучения:

- ▶ совершенствующиеся алгоритмы и архитектуры;
- ▶ доступные объёмы данных;
- ▶ ускорение обучения и вывода с помощью GPU.



< Предыдущий урок

Следующий урок >

12/17 Теоретическое резюме



Решаемые задачи: отображение X в Y:

- Текст, изображения, видео, звук, низкоразмерные данные.

Типы архитектур:

- Для распознавания и синтеза визуальных данных → **свёрточные** сети (CNN).
- Для распознавания и синтеза последовательностей → **рекуррентные** сети (RNN).
- Для улучшения качества синтеза → **состязательные** сети (GAN).

Факторы успеха: топология, данные, железо (GPU).

◀ Предыдущий урок

Следующий урок ▶

текущий урок

В основе современных фреймворков глубокого обучения лежит концепция **вычислительного графа** — удобного способа представления сложной функции в виде последовательности простых операций. Такое представление позволяет эффективно применять цепное правило и автоматически вычислять градиенты, что является ключевым механизмом обучения нейронных сетей.

Особое внимание уделим:

- ▶ принципам работы градиентного спуска,
- ▶ применению цепного правила в вычислительных графах,
- ▶ построению графа вычисления производных,
- ▶ связи этих концепций с реальными алгоритмами обучения.

В модуле вас ждёт не только теория, но и практика.

- ▶ разбор конкретных примеров (включая модель одного нейрона),
- ▶ работа в Google Colab,
- ▶ реализация простого вычислительного графа с использованием TensorFlow.

Завершает модуль практическое задание с самопроверкой, направленное на закрепление ключевых идей.

К концу модуля вы будете понимать, как фреймворки автоматически вычисляют производные и обучают модели, а также сможете самостоятельно реализовать базовые элементы этого процесса.

[← Предыдущий урок](#)

Следующий урок >

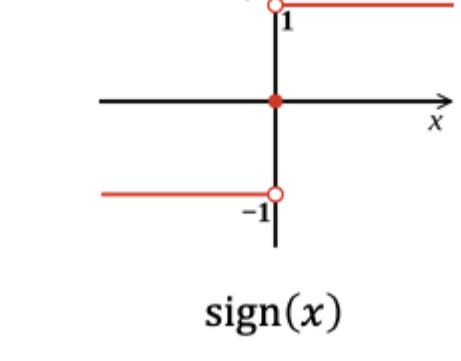


Прежде чем переходить к нейронным сетям, важно понять, из каких идей они вырастают. В этом уроке мы разберём базовые модели классификации — линейную классификацию и логистическую регрессию — и увидим, какие у них есть ограничения.

Это позволит понять, почему для решения более сложных задач требуются более мощные модели и как из простых линейных преобразований постепенно появляются нейронные сети.

Линейная классификация

Представьте, что у нас есть признаки $x = (x_1, x_2)$ и есть выборка положительных и отрицательных точек $y \in \{+1, -1\}$

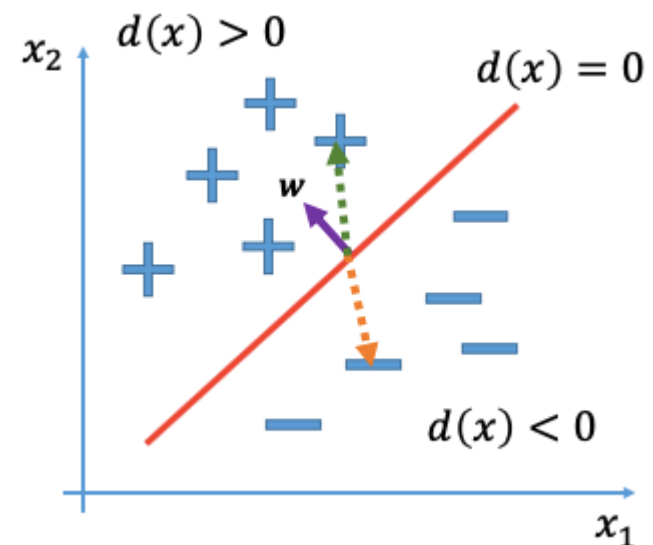


Нам нужно найти разделяющую гиперплоскость между ними. В данном случае это просто линия. Линия задается тремя коэффициентами:

$$d(x) = w_0 + w_1x_1 + w_2x_2$$

Нам нужно найти три коэффициента w , которые зададут линию. Далее мы можем взять точку x и понять, где она находится относительно линии: выше или ниже. Для этого нам нужно узнать знак линейной комбинации. Вектор весов w задаёт нормаль к нашей линии, то есть он перпендикулярен ей (фиолетовый вектор на графике ниже).

Линейная комбинация — это скалярное произведение и длина проекции какого-нибудь другого вектора на наш вектор w .



Поэтому проекция становится разных знаков. Из этих соображений мы делаем линейный классификатор. Наш алгоритм:

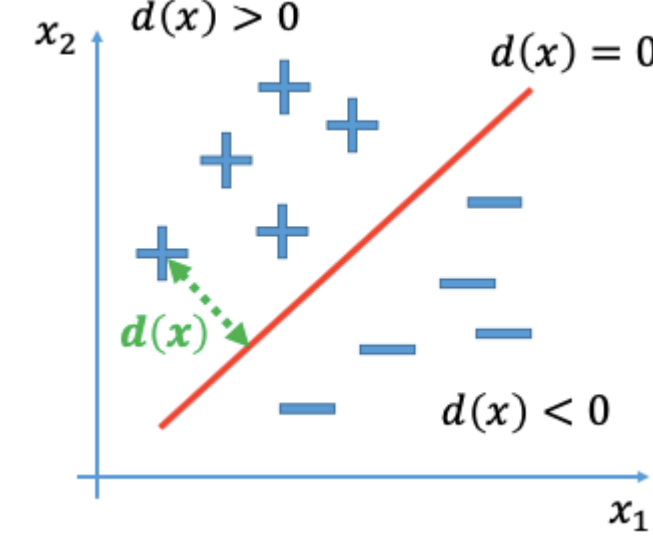
$$a(x) = \text{sign}(d(x))$$

Здесь появляется знак нашей линейной комбинации. Настроить линейный классификатор — значит, найти эти коэффициенты.

Логистическая регрессия

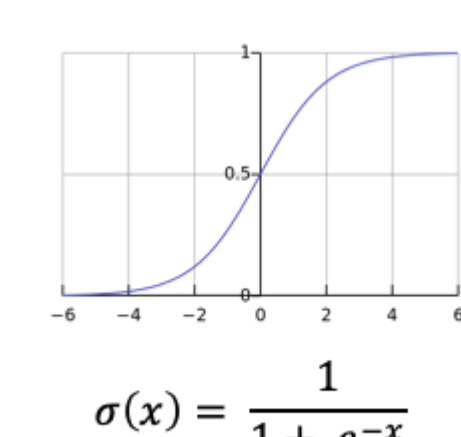
Она тоже решает задачу классификации, но в конце применяется не функция знака, а сигмоидная функция. Она превращает длину проекции в уверенность.

Уверенность и неуверенность появляется из-за краевых эффектов: на границе классов может быть какой-то шум, и в классификации точек, которые находятся рядом с красной разделяющей линией, мы не очень уверены.



А если мы уходим далеко от линии вглубь классов, то предполагается, что мы более уверены в этом предсказании. Сигмоида делает именно это — превращает длину проекции линейной комбинации в уверенность.

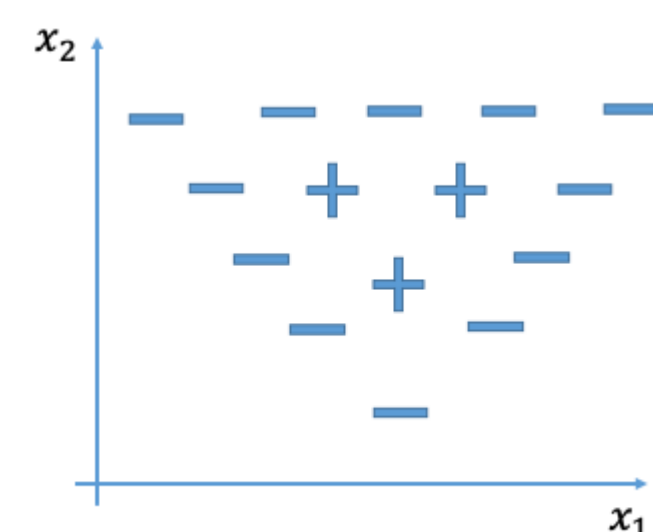
Сигмоида устроена не случайным образом:



Если длина проекции 0 (точка ровно на красной линии), то сигмоида даёт 0.5. Логистическая регрессия предсказывает вероятность положительного класса. Вероятность отрицательного будет единица минус предсказанная вероятность положительного класса.

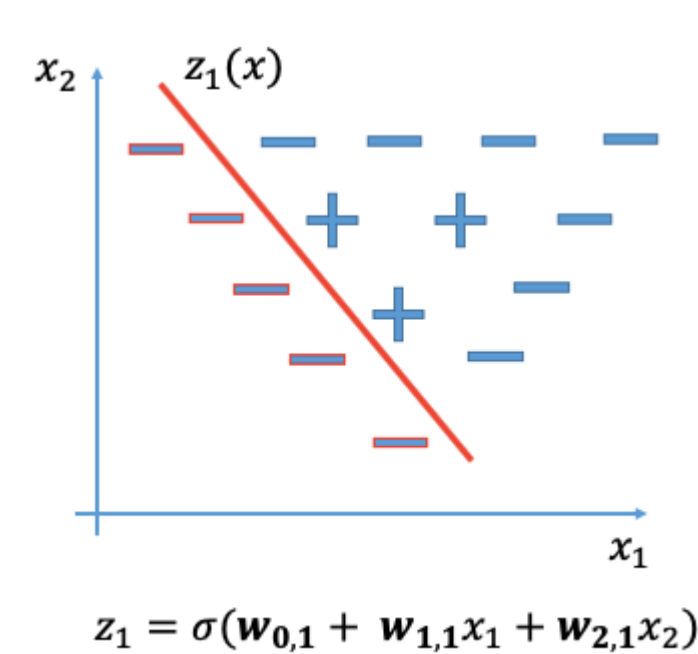
Другой пример

Представим, что у нас есть следующая задача:



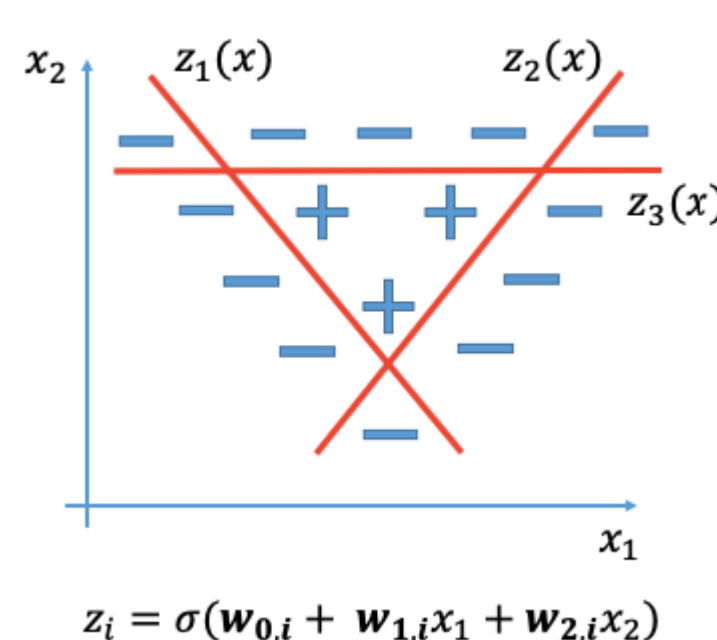
Чтобы решить подобную задачу, мы можем «подпереть» треугольник тремя линиями и сделать алгоритм, используя только логистическую регрессию.

Для начала мы отделим минусы слева и построим логистическую регрессию:



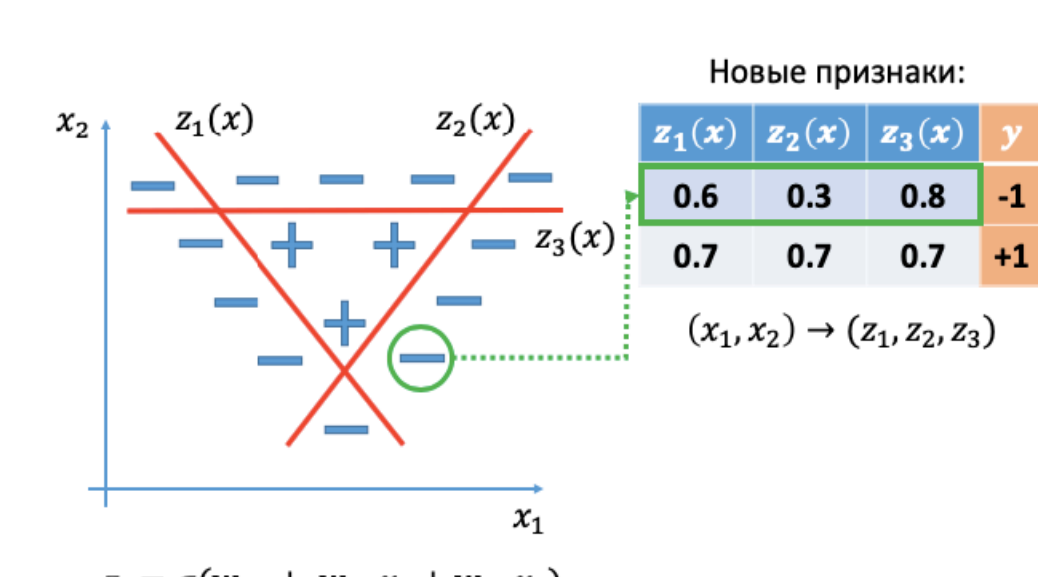
Эти данные подадим для обучения логистической регрессии и получим коэффициенты красной линии.

Отделим остальные минусы:



Все коэффициенты мы получили практически вручную. Сейчас у нас есть коэффициенты трёх логистических регрессий, каждая из которых решает свою маленькую задачу.

Теперь возьмём какую-нибудь точку и посмотрим, какие три предсказания дают эти линии в точке:



$$z_i = \sigma(w_{0,i} + w_{1,i}x_1 + w_{2,i}x_2)$$

Что делать с этими тремя значениями?

В координатах x_1 и x_2 эта задача не решается. Поэтому полученные коэффициенты мы можем рассматривать как новые координаты.

Получим три признака, каждый из которых говорит, где мы находимся относительно каждой стороны треугольника.

Давайте возьмём наш целевой признак y , добавим его в нашу новую таблицу, где наши новые признаки с предсказаниями, и попробуем решить её с новыми признаками с помощью линейной логистической регрессии. На новой выборке получим логистическую регрессию:

$$a(x) = \sigma(w_0 + w_1z_1(x) + w_2z_2(x) + w_3z_3(x))$$

Теперь она даёт нам некоторые коэффициенты и взвешивает уже не признаки, а предсказания. То, что мы получили — простейшая нейросеть.

Задание 2.1

0 points possible (ungraded)

Что определяет линейный классификатор?

- ☐ Расстояние до ближайшей точки обучающей выборки
- ☐ По какую сторону от разделяющей линии находится точка
- ☐ Вероятность принадлежности к классу
- ☐ Среднее значение признаков

Отправить

Задание 2.2

0 points possible (ungraded)

Что меняется при переходе от линейной классификации к логистической регрессии?

- ☐ Модель становится нелинейной
- ☐ Вместо знака используется сигмоида, и появляется вероятность
- ☐ Добавляется скрытый слой
- ☐ Используется другой набор признаков

Отправить